

MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC
OF KAZAKHSTAN INTERNATIONAL INFORMATION TECHNOLOGY
UNIVERSITY DEPARTMENT OF INFORMATION SYSTEM

LABORATORY WORK №4

TECHNICAL REPORT

Design, Architecture, and Implementation of a Full-Stack Single Page Application
with Session-Based Authentication

Done by: **Damirlan Zhetpissov**

Almaty 2026

1. Introduction and Theoretical Background

Modern web engineering relies heavily on the decoupling of client-side presentation layers and server-side business logic. The VÉLOX project implements a decoupled Single Page Application (SPA) architecture combined with a RESTful backend service.

From a theoretical standpoint, this separation of concerns (SoC) provides high modularity, independent scalability, and maintainability.

- **The Presentation Tier (Frontend):** Built using React, it functions as a state-driven view layer. React utilizes a Virtual DOM mechanism to optimize rendering performance by calculating minimal diffs before applying updates to the real browser DOM. Routing is handled client-side via react-router-dom, enabling seamless transitions without full-page reloads.
- **The Persistence and Logic Tier (Backend):** Built on Node.js utilizing the Express framework, the backend exposes RESTful endpoints. It handles business logic, input validation, and state persistence.

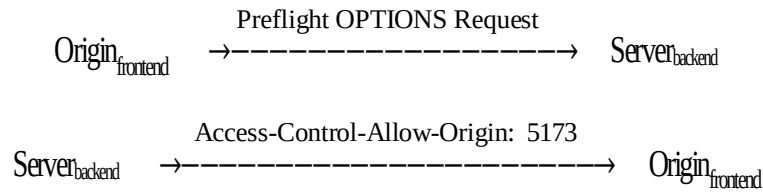
2. Architectural Design & Communication Protocols

2.1 Cross-Origin Resource Sharing (CORS) Mechanics

In this local development environment, the frontend runs on origin `http://localhost:5173` (served by the Vite development server), while the backend operates on `http://localhost:8080`.

According to the Same-Origin Policy (SOP) implemented by modern browsers, scripts running on one origin are restricted from interacting with resources from

another origin unless explicitly permitted. To bridge this gap, the backend implements the CORS middleware:

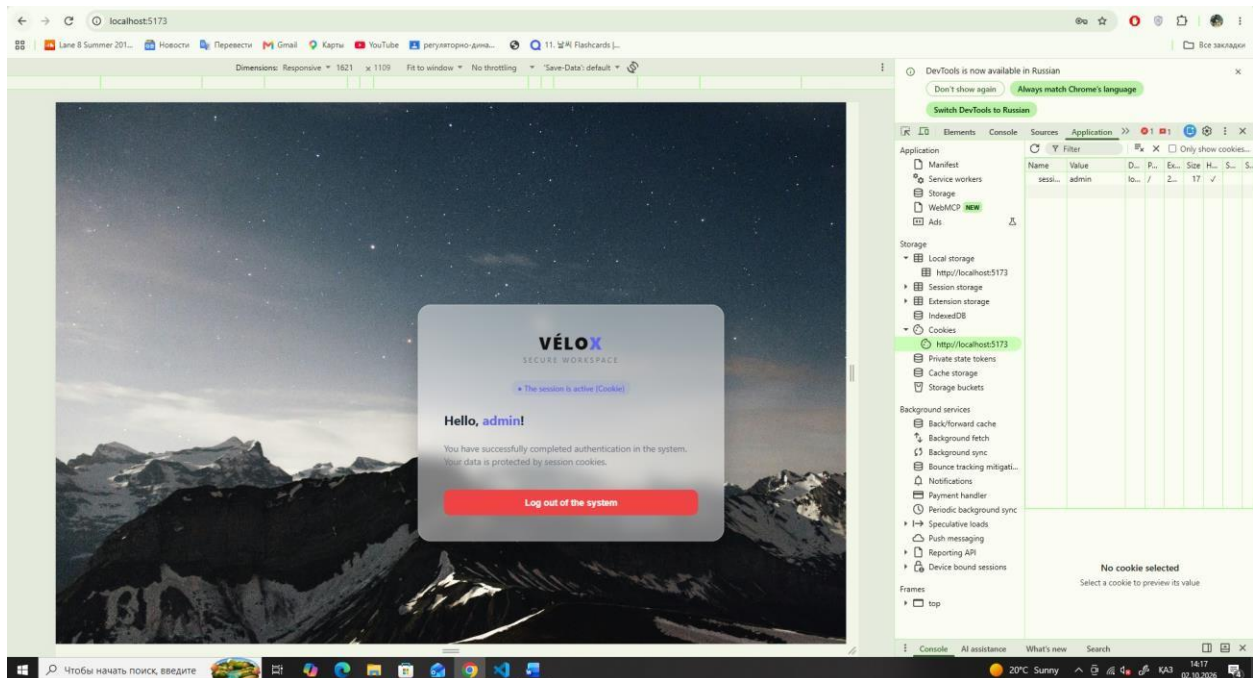


By configuring cors with `credentials: true`, the server allows the browser to include HTTP cookies and authorization headers across origins, which is critical for stateful sessions.

2.2 Cookie-Based State Management vs. Token-Based (JWT) Architecture

State management in distributed web systems typically follows two paradigms:

1. **Stateless Tokens (JWT):** Tokens are stored in local storage and sent via authorization headers. While scalable, they are vulnerable to Cross-Site Scripting (XSS) attacks if malicious scripts can read local storage.
2. **Stateful Session Cookies (HttpOnly):** Implemented in VÉLOX, the server generates a session identifier stored in an HttpOnly, SameSite cookie.
 - **Security Advantage:** HttpOnly cookies are entirely inaccessible to client-side JavaScript (`document.cookie` returns empty), mitigating the risk of token theft via XSS vulnerabilities.



Picture 1. Cookies

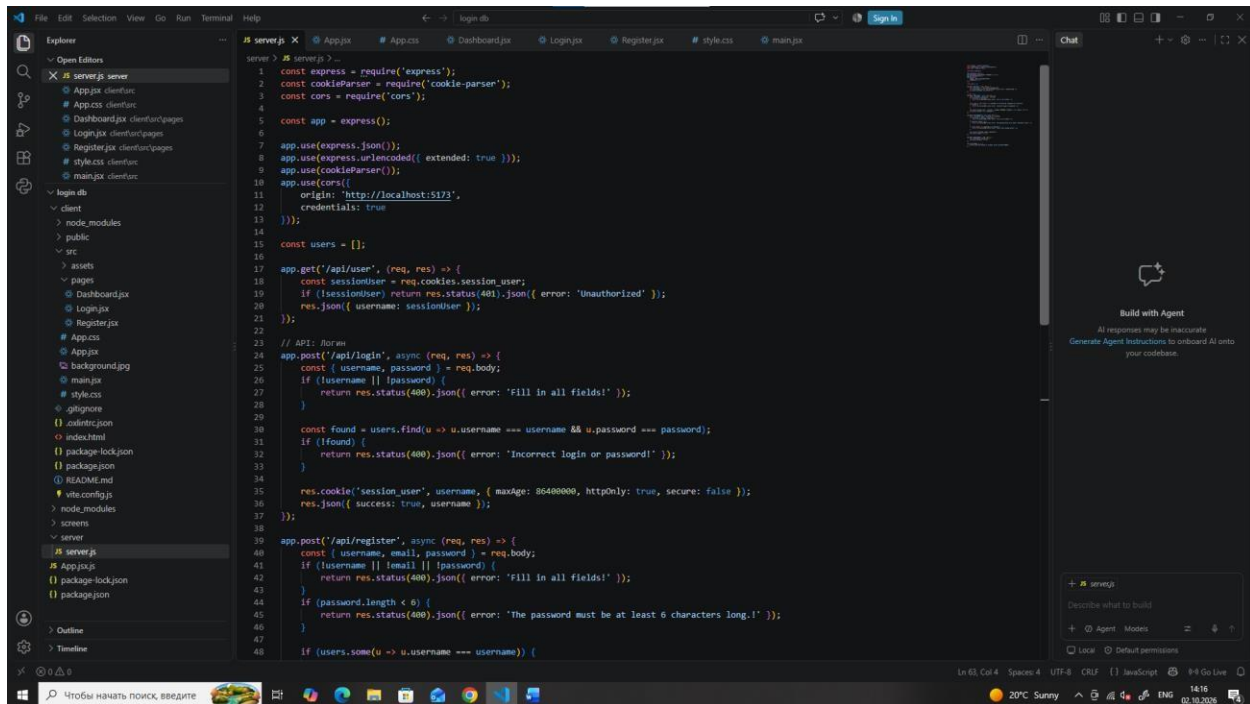
3. Implementation Details

3.1 Backend Architecture (server.js)

The server acts as an API gateway and session controller:

- **Middleware Stack:** Express parses incoming JSON payloads (`express.json()`), decodes URL-encoded structures, parses cookies (`cookie-parser`), and evaluates CORS policies.
- **Authentication Pipeline:**
 - Registration (`/api/register`) validates structural integrity (e.g., minimum password length constraints and uniqueness constraints).
 - Login (`/api/login`) validates credentials against the data store and issues an encrypted/signed session cookie (`session_user`).

- Session verification (/api/user) inspects incoming request cookies to authorize subsequent workspace requests.



Picture 2. Server.js backend

3.2 Frontend Architecture & Glassmorphism UI Theory

The user interface implements Glassmorphism, a design trend defined by transparency, multi-layered elements, and background blur effects.

- Technical realization via CSS:
 - backdrop-filter: blur(12px): Instructs the GPU to render a Gaussian blur on everything rendered *behind* the element.
 - rgba(255, 255, 255, 0.45): Provides a translucent white wash, allowing background elements (such as high-contrast wallpapers) to bleed through dynamically.

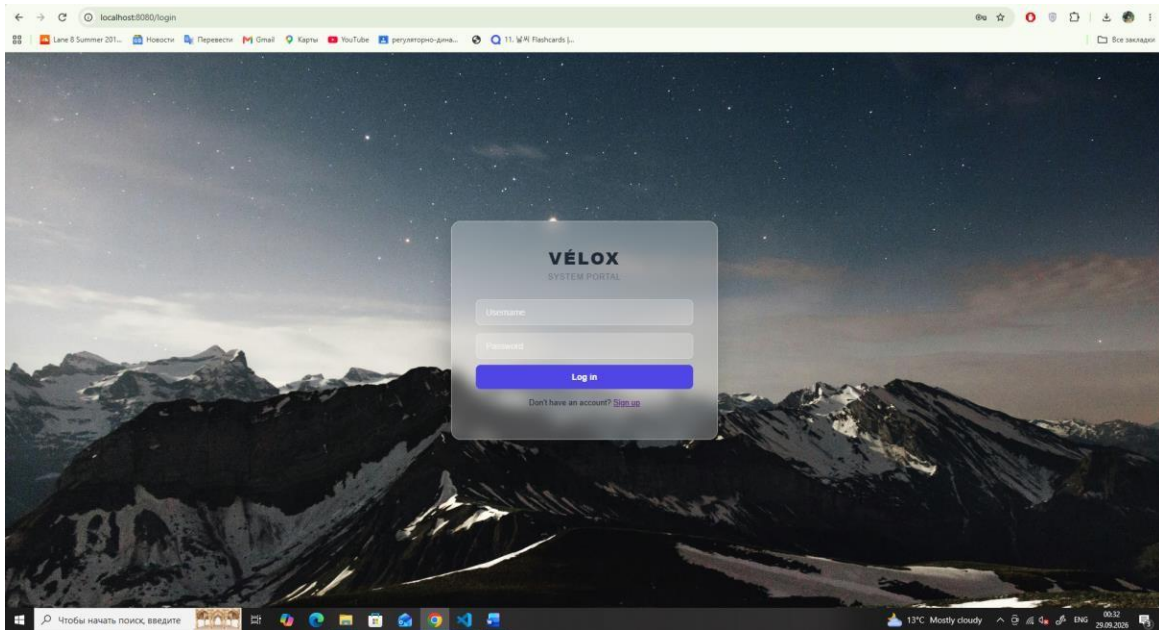
- Flexbox Layout Engine: Used for responsive alignment, centering the authentication card viewport-wide via flex containers (`display: flex; justify-content: center; align-items: center;`).

4. Testing, Verification, and Edge Cases

During the integration phase, several verification steps ensured system robustness:

- Network Interception & Preflight Validation: Using browser developer tools (Network tab), OPTIONS preflight requests were verified to ensure proper header reflection (`Access-Control-Allow-Credentials: true`).
- Route Guarding: Client-side routing ensures that unauthenticated users attempting to access the dashboard (/) are intercepted and redirected to the login sequence.
- Error Handling: Graceful error boundaries and JSON response parsing catch validation failures (e.g., empty fields, duplicate usernames) and display non-blocking inline error cards.
- Picture: Login Portal Interface

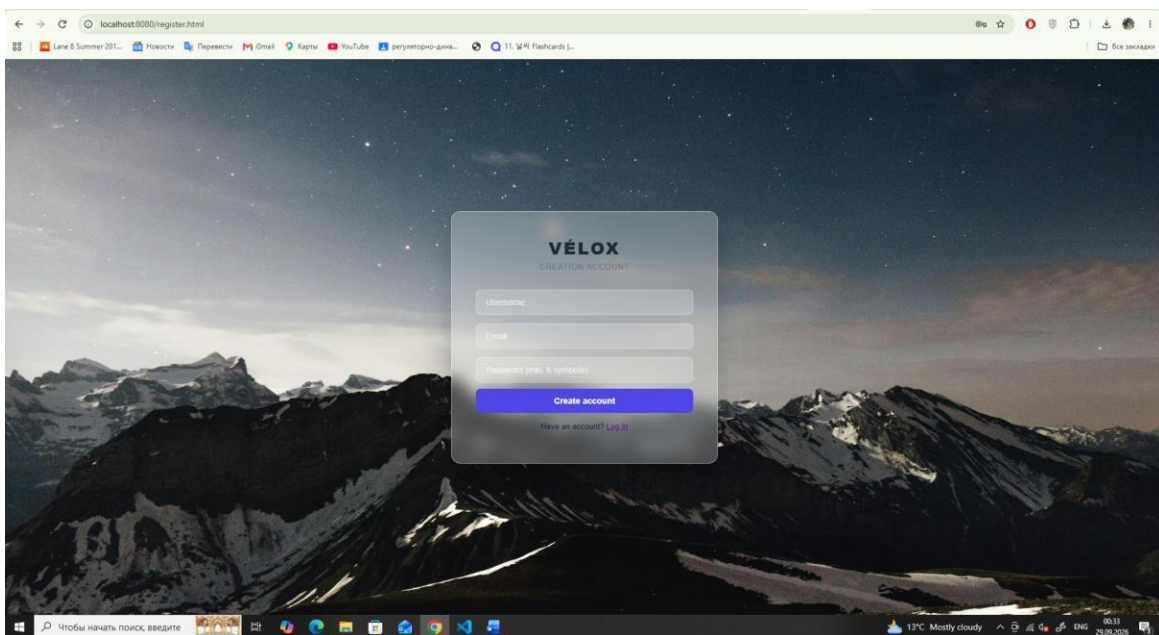
Capture the browser window displaying the login page with the dark starry/mountain background, the centered translucent glass card, VÉLOX branding logo, input fields for username and password, and the 'Log in' button.



Picture 3. Log in page

- Picture: Registration Form Interface

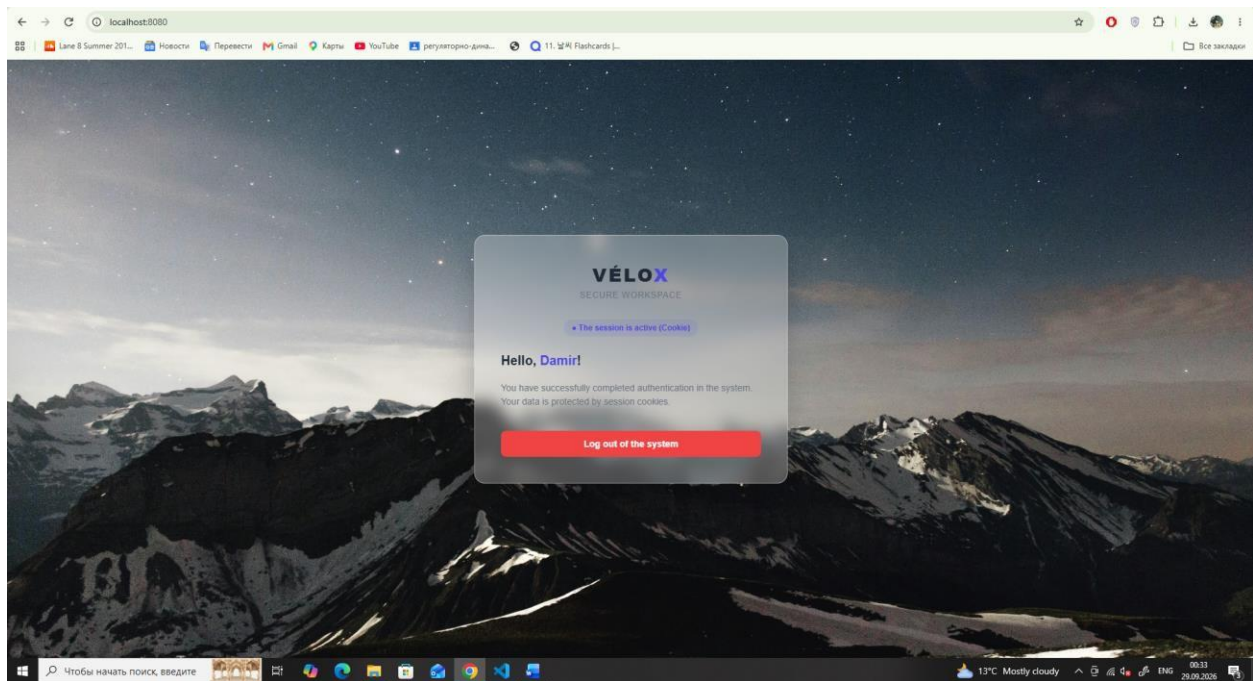
Capture the browser window showing the user registration interface where new accounts are created, demonstrating input validation fields and error-handling message containers.



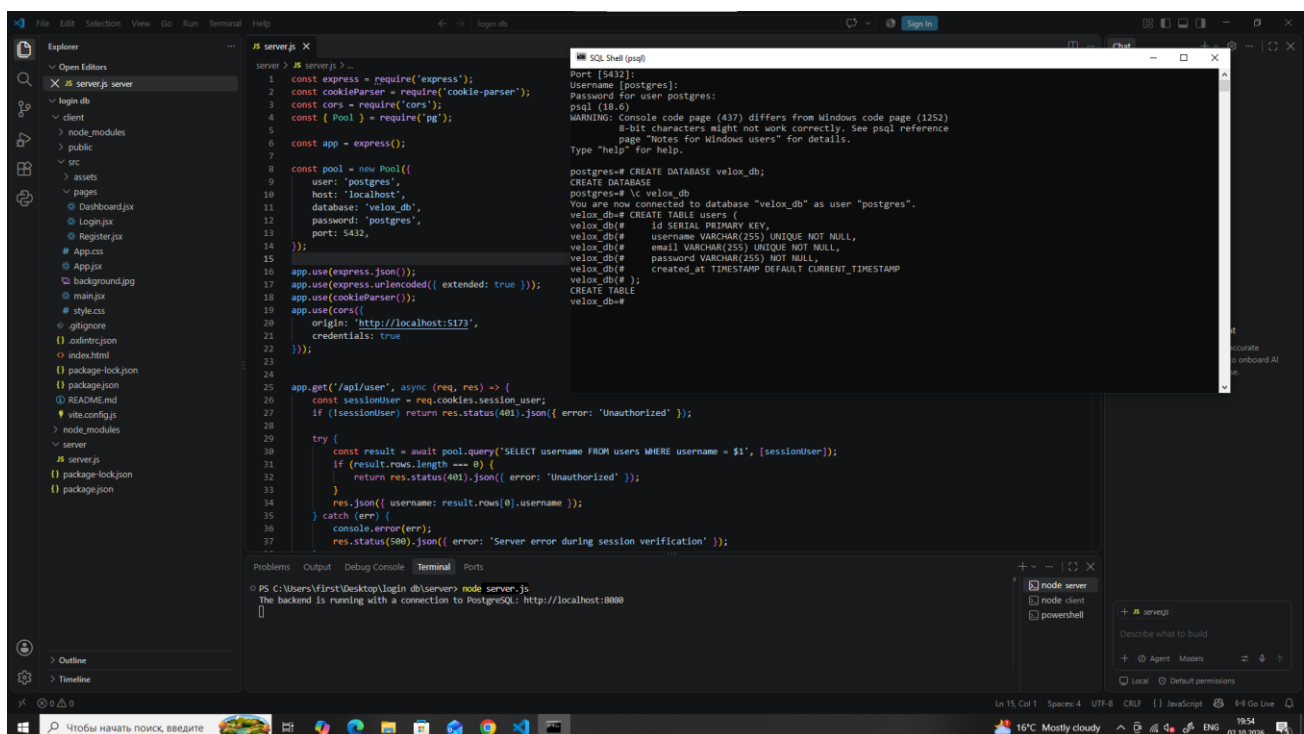
Picture 4. Register page

- Picture: Authenticated Dashboard / Welcome Screen

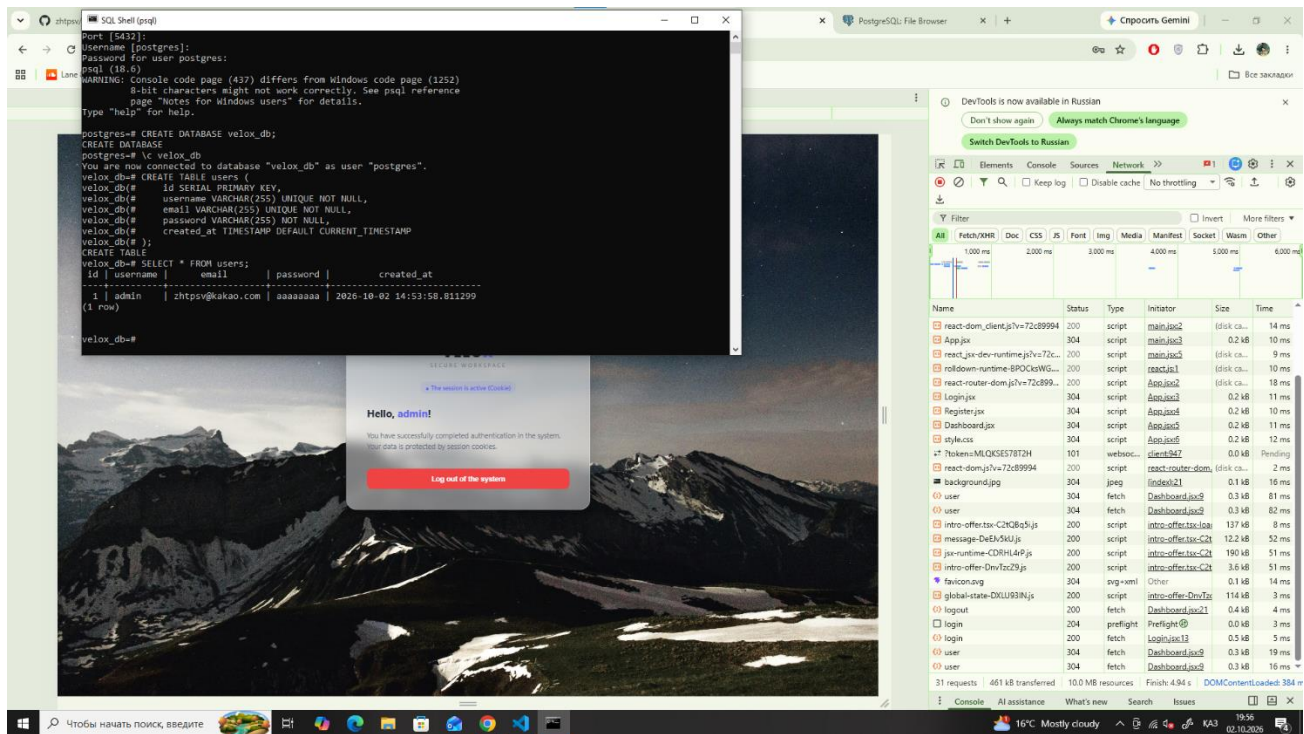
Capture the protected dashboard view after successful login, displaying the 'Welcome, [Username]!', the active session status cookie indicator badge, and the red 'Log out of the system' action button.



Picture 5. Welcome screen



Picture 6. Database screen



Picture 7. Database Network Traffic

5. Conclusion

The development and deployment of the **VÉLOX** web application successfully demonstrate the practical implementation of a robust, secure, and modern full-stack architecture. By decoupling the presentation tier—built with React and Vite—from the RESTful backend service powered by Node.js and Express, the project achieves high modularity, maintainability, and responsiveness.

Key technical achievements of this project include:

- **Secure State Management:** The successful implementation of HttpOnly and SameSite session cookies effectively mitigates common client-side vulnerabilities, such as Cross-Site Scripting (XSS), ensuring that sensitive user session identifiers remain isolated from malicious script execution.
- **Cross-Origin Integration:** Configuring explicit CORS policies with credentials enabled seamless, stateful communication across distinct development ports without compromising security or violating browser-enforced Same-Origin Policies.
- **Advanced User Interface Design:** The adoption of a Glassmorphism design system using CSS backdrop-filter and translucent color layers successfully bridges aesthetic innovation with high performance.

Future Scope: While the current version of VÉLOX fulfills all core functional and architectural requirements, future iterations can scale to include persistent database integration (such as PostgreSQL or MongoDB via connection pools), JSON Web Token (JWT) rotation mechanisms for multi-device tracking, and comprehensive automated testing pipelines using Jest and Cypress to ensure long-term code reliability.